DÍA MES AÑO NOTA /

POLYMORPHISM

Core concept y definition: Defines polymorphism (greek for "having many forms") as the ability to treat to treat related objects as the same type in client code while executing the correct specific behavior dynamically at runtime.

The problem with explicit Type-checking: Using explicit conditional checks (like instanceof and casting) leads to verbose, error-pone code that becomes a maintenace nightmare whenever new subtypes are introduced.

Polymorphic solution: Solved by establishing a common abstract interface or superclass (e.g., ChessPiece), implementing required methods in concrete subclasses (Pawn, Rook), and depending strictly on the superclass in client code

Trade-offs y extensibility: Improve extensibility because new behaviors can be added via subclasses without modifying existing client code, though the exact runtime behavior is deferred until execution time.

Abstraction y java implementation: focuses only on relevant parts while ignoring unnecesary details; software systems should identify cros-domain similarities and delegate specific differences to concrete subclasses, covering method overriding, overloading, and generalization/specialization hierarchies (e.g., FarmAnimal and Chicken)

Scanned with CamScanner

DÍA MES AÑO NOTA /

INTERFACES

Review of localization and Encapsulation: Recaps the localization of design decisions, encapsulation principales, and their importance in software architecture.

Definition of Abstraction: Defined both as verb (creating an interface or class definition to expose necessary component details wile hiding implementations) and a noun (the resulting description leaving out unnecessary details).

Contextual Modeling: Demonstrates that identical concepts (such as a chicken) or a farm animal) requie entirely different properties depending on the specific application context, such as a farm information system versus a coop simulator.

Abstract classes and interface in Java:

Convers Java keywords such as abstract (classes that cannot be instantiated, and abstract methods that derived classes must implement).

ABSTRACTION

Background y Philosophy: Explores abstraction as bringing relevant details to the forefront wile suppressing irrelevant information within a give context, as well as the reification of conceptual distinctions.

System-Specific Abstractions: Illustrates how properties modeled for a generic entity like a person vary dastically across different systems, such as a football tournament tracking system versus an airline reservation system.

Scanned with CamScanner

DÍA MES AÑO NOTA /

common pitfalls: Identifies leaky abstractions (where components rely on unstated implementation details) and over-abstraction (where too much information is omitted leaving users with insufficient control).

Adherence criteria y case studies: Outlines criteria for high-quality software abstractions - meaningful/context aware identifiers, explicit declarations, sufficiency, and conciseness - applied to contact lists, ant-colony simulators banking libraries, and geometric shape libraries.

Scanned with CamScanner